PROMPT ENGINEERING PROJECT 2

Chain-of-Thought (CoT) Logic Engine

Submitted By

Pradeep Kumar

Branch

B.Tech CSE-AIML

Organization

DecodeLabs Industrial Training Program

Academic Year

2026

ABSTRACT

Large Language Models (LLMs) are capable of solving complex reasoning tasks; however, they often produce incorrect answers due to skipped reasoning steps and hallucinations. Chain-of-Thought (CoT) Prompting is an advanced prompt engineering technique that forces AI models to explicitly show intermediate reasoning before producing a final answer.

This project presents a Chain-of-Thought Logic Engine designed to improve reasoning accuracy through structured problem solving, self-correction mechanisms, verification phases, and logic trap detection. The system guides the model through multiple reasoning stages, reducing hallucinations and improving answer reliability.

Keywords: Prompt Engineering, Chain-of-Thought, Artificial Intelligence, Large Language Models, Reasoning Engine, Self-Correction, Logic Validation.

INTRODUCTION

Prompt Engineering has emerged as one of the most important skills in modern Artificial Intelligence. Instead of retraining a model, prompt engineering enables developers to guide model behavior using carefully designed instructions.

Chain-of-Thought Prompting is a reasoning strategy that requires an AI system to break a problem into smaller logical steps before arriving at a final answer. Research has shown that CoT prompting significantly improves reasoning performance in arithmetic, logic, and decision-making tasks.

This project focuses on designing a Logic Engine that enforces structured reasoning and self-verification.

OBJECTIVES

The objectives of this project are:

1. Understand Chain-of-Thought prompting.
2. Design structured reasoning workflows.
3. Implement self-correction mechanisms.
4. Reduce hallucinations in AI-generated responses.
5. Test reasoning performance on logic traps and riddles.
6. Improve answer reliability through verification.

PROBLEM STATEMENT

Standard AI systems often generate answers directly without explaining their reasoning process. This can lead to:

- Logical errors
- Incorrect assumptions
- Hallucinations
- Arithmetic mistakes

The challenge is to force the model to think systematically before generating an answer.

THEORY

Chain-of-Thought Prompting

Chain-of-Thought Prompting encourages the AI model to generate intermediate reasoning steps before reaching a final conclusion.

Example:

Problem: A bat and a ball cost ₹110. The bat costs ₹100 more than the ball.

Reasoning:

Bat = Ball + 100

(Ball + 100) + Ball = 110

2 × Ball = 10

Ball = 5

Bat = 105

Final Answer: Ball = ₹5 Bat = ₹105

This approach significantly improves reasoning accuracy.

Self-Correction

Self-Correction requires the model to review its own reasoning process.

The system checks:

- Arithmetic calculations
- Logical consistency
- Contradictions
- Missing assumptions

This phase helps identify and correct mistakes before producing the final answer.

Logic Trap Detection

Logic traps are problems designed to exploit common reasoning mistakes.

Example:

A farmer has 17 sheep. All but 9 die.

Correct Answer: 9 sheep remain.

The Logic Engine is designed to identify such traps and avoid common mistakes.

SYSTEM ARCHITECTURE

Input Problem ↓ Fact Extraction ↓ Assumption Analysis ↓ Step-by-Step Reasoning ↓ Self-Correction ↓ Verification ↓ Final Answer

The architecture ensures transparency and traceability throughout the reasoning process.

METHODOLOGY

The project follows six stages:

Phase 1: Fact Extraction

The system extracts all explicit information from the problem.

Phase 2: Assumption Analysis

Hidden assumptions and ambiguities are identified.

Phase 3: Step-by-Step Reasoning

The problem is solved using detailed logical steps.

Phase 4: Self-Correction

The generated reasoning is reviewed for mistakes.

Phase 5: Verification

The solution is independently verified.

Phase 6: Final Answer

A concise and accurate answer is produced.

IMPLEMENTATION

Tools Used:

- Python
- Streamlit
- Prompt Engineering
- GitHub

Modules Developed:

1. Logic Engine
2. Prompt Loader
3. Reasoning Validator
4. Self-Correction Engine
5. Streamlit Interface

TEST CASES

Test Case 1

Problem:

A bat and a ball cost ₹110. The bat costs ₹100 more than the ball.

Answer:

Ball = ₹5 Bat = ₹105

Test Case 2

Problem:

A farmer has 17 sheep. All but 9 die.

Answer:

9 sheep remain.

Test Case 3

Problem:

A lily pad doubles in size every day. It covers the pond on Day 48.

Answer:

Half the pond was covered on Day 47.

RESULTS

The Logic Engine successfully:

- Generated structured reasoning.
- Detected logic traps.
- Reduced reasoning errors.
- Improved answer transparency.
- Enhanced reliability through verification.

ADVANTAGES

1. Reduces hallucinations.
2. Improves logical reasoning.
3. Enhances transparency.
4. Provides explainable AI outputs.
5. Supports educational applications.

APPLICATIONS

1. AI Tutors
2. Educational Platforms
3. Decision Support Systems
4. Automated Reasoning Systems
5. Intelligent Assistants
6. Enterprise AI Workflows

LIMITATIONS

1. Longer responses due to reasoning chains.
2. Increased token usage.
3. Performance depends on prompt quality.

FUTURE SCOPE

Future enhancements include:

- Multi-agent reasoning systems.
- Debate-based reasoning.
- Retrieval-Augmented Generation (RAG).
- Fine-tuned reasoning models.
- Real-time enterprise deployment.

CONCLUSION

This project demonstrates the practical application of Chain-of-Thought Prompting for solving complex reasoning tasks. By enforcing structured reasoning, self-correction, and verification phases, the Logic Engine significantly improves the reliability and transparency of AI-generated answers. The project highlights how prompt engineering can be used to guide large language models toward more accurate and explainable reasoning processes.

REFERENCES

1. DecodeLabs Prompt Engineering Training Material.
2. OpenAI Prompt Engineering Documentation.
3. Google Research on Chain-of-Thought Prompting.
4. Large Language Model Reasoning Research Papers.
5. Artificial Intelligence and Explainable AI Literature.